

Assignment: Instruction Tuning and Direct Preference Optimization on a Small Language Model

Course Staff

Abstract

This assignment introduces the post-training pipeline used to turn a pretrained language model into a more useful assistant. You will work with a small pretrained model, `Qwen/Qwen2.5-0.5B`, and train it in two stages: supervised fine-tuning (SFT) on instruction-response demonstrations, followed by Direct Preference Optimization (DPO) on pairwise preference data. You will implement the main SFT and DPO training components yourself, run the experiments on Kaggle T4 GPUs, generate responses on a fixed ALPACAEVAL subset, and evaluate model behavior using pairwise judgments from Kaggle model APIs. The goal is not to produce a state-of-the-art assistant, but to understand the mechanics, tradeoffs, and evaluation challenges of modern LLM alignment.

Contents

1	Learning Objectives	3
2	High-Level Background	3
2.1	Pretraining	3
2.2	Supervised Fine-Tuning	3
2.3	RLHF	4
2.4	Direct Preference Optimization	4
3	What You Will Build	4
4	Automated Checker Interface	5
5	Compute Environment	6
5.1	Kaggle Accelerator	6
5.2	Dependency Guidance	6
5.3	Recommended Repository Layout	6
5.4	Kaggle Workflow	7
6	Model and Data	7
6.1	Model	7
6.2	SFT Data	7
6.3	Preference Data	7

6.4	Evaluation Prompts	8
7	Task 1: Repository and Kaggle Setup	8
8	Task 2: Baseline Generation	8
9	Task 3: Supervised Fine-Tuning	9
9.1	Implementation Requirements	9
9.2	Suggested Hyperparameters	10
10	Task 4: DPO From Scratch	10
10.1	Implementation Requirements	10
10.2	Suggested Hyperparameters	11
10.3	Metrics	11
11	Task 5: Generate Evaluation Outputs	12
12	Task 6: Evaluation With Kaggle Models	13
13	Task 7: Final Report	14
13.1	Required Tables	14
13.2	Required Qualitative Analysis	14
14	Expected Runtime	15
15	Submission Checklist	15
16	Grading Rubric	16
17	Academic Integrity and Collaboration	16
18	Extension Task: The Idiot Sandwich — TRL, PEFT, and Style Preference Tuning	16
18.1	Model	16
18.2	Dataset	17
18.3	Training Method	17
18.4	Suggested LoRA Configuration	17
18.5	Hyperparameter Sweep	18
18.6	Evaluation	18
18.7	Report Requirements for the Extension	19
18.8	What to Compare Against the Main Assignment	19
19	Resources	20

1 Learning Objectives

By the end of this assignment, you should be able to:

	Objective
✓	Explain the distinction between pretraining, supervised fine-tuning, RLHF, and DPO.
✓	Implement a response-masked supervised fine-tuning loop for a causal language model.
✓	Implement the DPO loss from pairwise preference data without using TRL's DPOTrainer.
✓	Train a small language model under realistic GPU and disk constraints.
✓	Design and run an evaluation pipeline based on fixed prompts, saved generations, and pairwise model judgments.
✓	Analyze alignment results critically, including failure cases, noisy judgments, and possible alignment tax.

2 High-Level Background

2.1 Pretraining

Modern language models are first trained with a self-supervised next-token prediction objective. Given a sequence of tokens $x_1, \dots, x_T$, the model learns to maximize

$$\sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}). \quad (1)$$

This stage teaches the model broad linguistic, factual, and reasoning patterns from large text corpora. However, a pretrained base model is not necessarily a good assistant. It may continue text, imitate undesired formats, fail to follow instructions reliably, or produce unsafe content.

In this assignment, you will *not* pretrain a model. You will start from a pretrained base model, Qwen/Qwen2.5-0.5B, and study post-training.

2.2 Supervised Fine-Tuning

Supervised fine-tuning (SFT), also called instruction tuning, trains the model on examples of the form:

instruction $\rightarrow$ **response**.

The model is trained with teacher forcing to assign high probability to the target response. A key implementation detail is that the loss should usually be applied only to response tokens, not to the prompt tokens. If x is the instruction prompt and y is the target response, the SFT objective is

$$\mathcal{L}_{\text{SFT}}(\theta) = -\log \pi_{\theta}(y \mid x). \quad (2)$$

SFT teaches a base model to follow an instruction-response format and to imitate high-quality demonstrations.

2.3 RLHF

Reinforcement Learning from Human Feedback (RLHF) typically involves three stages:

1. Train an SFT model from demonstrations.
2. Train a reward model from pairwise human preferences.
3. Optimize the language model policy against the reward model, often using PPO with a KL penalty.

RLHF is powerful but complex. It requires reward modeling, online sampling, reinforcement learning machinery, and careful stabilization.

You will *not* implement PPO or full RLHF in this assignment. Instead, you will implement DPO, a simpler preference-learning method.

2.4 Direct Preference Optimization

DPO uses pairwise preference data directly. Each example contains an instruction x , a preferred response y_w , and a rejected response y_l . DPO compares the trainable policy π_θ with a frozen reference model π_{ref} , usually the SFT model.

The per-example DPO loss is

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right] \right), \quad (3)$$

where β controls how strongly the policy is allowed to move away from the reference model.

Equivalently,

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left[(\log \pi_\theta(y_w | x) - \log \pi_\theta(y_l | x)) - (\log \pi_{\text{ref}}(y_w | x) - \log \pi_{\text{ref}}(y_l | x)) \right] \right). \quad (4)$$

DPO in One Sentence

DPO does not require training an explicit reward model and does not require online sampling during training. It only requires computing conditional log probabilities for chosen and rejected responses.

3 What You Will Build

You will build a complete small-model alignment pipeline:

1. Create a reproducible GitHub repository.
2. Set up Kaggle notebooks using T4 GPUs.
3. Generate baseline responses from Qwen/Qwen2.5-0.5B on an ALPACAEVAL subset.
4. Implement and run SFT.
5. Implement and run DPO from scratch.
6. Generate outputs from the base, SFT, and DPO models.

7. Build pairwise comparison files.
8. Evaluate using Kaggle model-based pairwise judgments.
9. Submit a complete written report.

Library Boundary

You may use [Hugging Face Transformers](#) to load models and tokenizers. You may use PyTorch utilities such as `DataLoader`, optimizers, schedulers, and AMP. For the main SFT and DPO implementation, you may **not** use Hugging Face `Trainer`, `TRL DPOTrainer`, or a prebuilt DPO training loop.

4 Automated Checker Interface

You may organize your code however you like, but your repository must include a file named `submission_adapter.py` at the repository root. The checker imports this file and calls a small fixed API. Each adapter function should call into your own SFT and DPO implementation.

```
def build_sft_batch(tokenizer, prompts, responses, max_length): ...

def compute_response_logprobs(model, tokenizer, prompts, responses, max_length): ...

def compute_sft_loss(model, tokenizer, prompts, responses, max_length): ...

def compute_dpo_loss(
    policy_model,
    reference_model,
    tokenizer,
    prompts,
    chosen_responses,
    rejected_responses,
    beta,
    max_length,
): ...
```

The adapter exists so that you are free to choose your own internal structure while the checker can still test the important behavior. The checker will use a tiny toy tokenizer and toy causal language model, not `Qwen/Qwen2.5-0.5B`, so the tests run quickly and focus on correctness.

Your adapter must satisfy the following contract:

- ▶ Preserve the input row order in all returned tensors and vectors.
- ▶ Compute losses and log probabilities over response tokens only.
- ▶ Include an EOS token in the response when the tokenizer provides one.
- ▶ Mask prompt tokens and padding tokens with `-100` for SFT labels.
- ▶ Return scalar mean losses for SFT and DPO.
- ▶ Keep the DPO reference model frozen; reference-model computations should not produce gradients.

You can run the public checker locally with:

```
python -m pip install -r autograder/requirements.txt
PYTHONPATH=. pytest tests/test_adapter_correctness.py
```

5 Compute Environment

5.1 Kaggle Accelerator

Use a Kaggle notebook with the GPU T4 x2 accelerator. You should verify that two GPUs are visible:

```
nvidia-smi
```

A typical output should show two Tesla T4 GPUs with roughly 15 GiB of memory each.

5.2 Dependency Guidance

Kaggle already includes a CUDA-compatible PyTorch build. Do not reinstall PyTorch unless you have a specific reason. A safe install pattern is:

```
python -m pip install -q "transformers>=4.43.0,<5" "datasets>=2.18,<5" \
    huggingface_hub accelerate tqdm pyyaml sentencepiece safetensors
python -m pip install -q -e . --no-deps
```

Your repository should include a minimal `requirements.txt` and `pyproject.toml`. The requirements should not pin torch. A suitable `requirements.txt` is:

```
transformers>=4.43.0,<5
datasets>=2.18,<5
huggingface_hub
accelerate
tqdm
pyyaml
sentencepiece
safetensors
```

5.3 Recommended Repository Layout

You should create a GitHub repository with a clear structure. One possible layout is:

```
qwen-dpo-alignment/
  requirements.txt
  pyproject.toml
  README.md
  src/
    your_package/
      data.py
      modeling.py
      prompts.py
      utils.py
  scripts/
    train_sft.py
    train_dpo.py
```

```

generate_alpaca.py
make_alpaca_subset.py
build_pairwise_eval.py
judge_pairwise_kaggle.py
notebooks/
  kaggle_train_only.ipynb
  kaggle_generate_outputs.ipynb
  kaggle_eval_only.ipynb

```

You may choose a different structure, but your code must be reproducible and documented.

5.4 Kaggle Workflow

A practical workflow is:

1. Develop and push your code to GitHub.
2. Clone the repository in Kaggle.
3. Run training in one notebook.
4. Save the Kaggle notebook version so that outputs are preserved.
5. Attach the training notebook outputs as an input dataset to a separate generation notebook.
6. Attach generation outputs as an input dataset to a separate evaluation notebook.

This separation avoids rerunning expensive training when debugging evaluation.

6 Model and Data

6.1 Model

Use Qwen/Qwen2.5-0.5B as the base model. This model is small enough to train on Kaggle T4 GPUs, while still being large enough to show meaningful differences between base, SFT, and DPO behavior.

6.2 SFT Data

Use an instruction-response dataset such as UltraChat-style data. Each example should be converted into a single-turn instruction and response.

Recommended run size:

Split	Examples	Purpose
Train	30,000	SFT optimization
Validation	1,000	validation loss

6.3 Preference Data

Use pairwise preference data such as the Anthropic Helpful–Harmless dataset. Each example should contain:

instruction, chosen, rejected.

You should filter out multi-turn examples if your preprocessing assumes single-turn prompts.

Recommended run size:

Split	Pairs	Purpose
Train	15,000	DPO optimization
Validation	1,000	reward accuracy and margin

6.4 Evaluation Prompts

Use a fixed ALPACAEVAL subset with a deterministic seed. Recommended size:

Subset	Prompts
Default	300
Larger optional run	500

Save the exact subset used for all models.

7 Task 1: Repository and Kaggle Setup

Create a GitHub repository and a Kaggle notebook workflow. Your setup should:

1. Clone your repository in Kaggle.
2. Install dependencies without reinstalling PyTorch.
3. Confirm that two T4 GPUs are available.
4. Print package versions for PyTorch, Transformers, and Datasets.

Deliverables. Submit your GitHub URL, Kaggle setup logs, and a short note describing your environment.

8 Task 2: Baseline Generation

Create a fixed ALPACAEVAL subset and generate baseline responses from Qwen/Qwen2.5-0.5B.

Your generation script should save a JSON file with entries containing at least:

```
{
  "id": "...",
  "instruction": "...",
  "output": "...",
  "generator": "qwen2.5-0.5b-base",
  "dataset": "alpaca_eval"
}
```

Use greedy decoding for the main comparison unless you justify another choice.

Deliverables. Submit the subset file, baseline generation file, generation throughput, and five qualitative examples.

9 Task 3: Supervised Fine-Tuning

9.1 Implementation Requirements

Implement your own SFT training loop. Your implementation should include:

1. Prompt formatting for instruction-response examples.
2. Response-only loss masking.
3. Padding and attention masks.
4. Gradient accumulation.
5. Mixed precision using `fp16` autocast on T4 GPUs.
6. FP32 trainable weights when using gradient scaling.
7. Gradient clipping.
8. Validation loss.
9. Saving `best` and `final` checkpoints.

Do not save frequent full intermediate checkpoints on Kaggle unless you have enough disk space. Saving only `best` and `final` is sufficient.

9.2 Suggested Hyperparameters

Hyperparameter	Value
Model	Qwen/Qwen2.5-0.5B
Context length	512
Train examples	30,000
Validation examples	1,000
Per-device batch size	2
Gradient accumulation	16
Effective batch size	32
Learning rate	2×10^{-5}
Warmup	3%
Schedule	cosine decay
Weight decay	0.1
Gradient clipping	1.0
Epochs	1

SFT Deliverables

Submit SFT training logs, final validation loss, runtime, checkpoint path, and a brief discussion of qualitative changes from the base model.

10 Task 4: DPO From Scratch

10.1 Implementation Requirements

Implement DPO yourself. Your implementation should include:

1. Loading the SFT model as the trainable policy.
2. Loading a frozen copy of the SFT model as the reference model.
3. Computing response-only log probabilities for chosen and rejected responses.
4. Computing the DPO loss.

5. Tracking validation DPO loss, reward accuracy, and reward margin.
6. Saving the best checkpoint according to validation reward accuracy.

Kaggle Memory Tip

A memory-friendly setup on Kaggle T4x2 is to place the policy on `cuda:0` and the frozen reference on `cuda:1`. The trainable policy should remain in fp32 when using gradient scaling; the frozen reference may be loaded in fp16.

10.2 Suggested Hyperparameters

Hyperparameter	Value
Initial policy	SFT checkpoint
Reference model	frozen SFT checkpoint
Train pairs	15,000
Validation pairs	1,000
Context length	512
Per-device batch size	1
Gradient accumulation	32
Effective batch size	32
Learning rate	1×10^{-6}
β	0.1
Optimizer	RMSprop or AdamW
Epochs	1

10.3 Metrics

For each validation pair, define the implicit rewards:

$$r_w = \beta (\log \pi_\theta(y_w | x) - \log \pi_{\text{ref}}(y_w | x)), \quad (5)$$

$$r_l = \beta (\log \pi_\theta(y_l | x) - \log \pi_{\text{ref}}(y_l | x)). \quad (6)$$

Reward accuracy is the fraction of validation examples where

$$r_w > r_l. \quad (7)$$

Reward margin is the mean value of

$$r_w - r_l. \quad (8)$$

A reward accuracy near 0.5 with margin near 0 suggests little preference signal. Higher reward accuracy with a positive margin suggests the policy is learning to prefer chosen responses.

DPO Deliverables

Submit DPO training logs, validation DPO loss, reward accuracy, reward margin, checkpoint path, and a short discussion of training stability.

11 Task 5: Generate Evaluation Outputs

Before launching full-subset generation, run a small sanity check on 2–5 prompts. Inspect the responses manually to confirm that the checkpoint path, tokenizer, prompt format, EOS behavior, and generation length are reasonable.

```
python scripts/generate_alpaca.py \
  --model_name_or_path outputs/sft-qwen-0.5b/best \
  --subset_path data/alpaca_eval/alpacaeval_300_seed42.json \
  --output_path outputs/eval/sanity_sft_5.json \
  --generator_name qwen2.5-0.5b-sft-sanity \
  --batch_size 2 \
  --max_new_tokens 256 \
  --limit 5 \
  --device cuda:0
```

After the sanity check passes, generate responses on the same ALPACAEVAL subset for:

1. Base model.
2. SFT model.
3. DPO model.

Using Both Kaggle T4 GPUs for Generation

Generation is embarrassingly parallel over prompts. On Kaggle T4x2, split the prompt subset into two interleaved shards and run one process on `cuda:0` and one process on `cuda:1`. Merge the shard outputs back into the original subset order before building pairwise files.

A two-GPU run for one model can be launched as two simultaneous processes:

```
python scripts/generate_alpaca.py \
  --model_name_or_path outputs/dpo-qwen-0.5b/best \
  --subset_path data/alpaca_eval/alpacaeval_300_seed42.json \
  --output_path outputs/eval/dpo_shard0.json \
  --generator_name qwen2.5-0.5b-dpo \
  --batch_size 8 \
  --max_new_tokens 512 \
  --device cuda:0 \
  --num_shards 2 \
  --shard_index 0 &
```

```
python scripts/generate_alpaca.py \  
  --model_name_or_path outputs/dpo-qwen-0.5b/best \  
  --subset_path data/alpaca_eval/alpacaeval_300_seed42.json \  
  --output_path outputs/eval/dpo_shard1.json \  
  --generator_name qwen2.5-0.5b-dpo \  
  --batch_size 8 \  
  --max_new_tokens 512 \  
  --device cuda:1 \  
  --num_shards 2 \  
  --shard_index 1 &
```

wait

Your notebook may wrap this in Python with `subprocess.Popen`, then merge the two JSON files by prompt ID. The provided Kaggle generation notebook includes a helper for this pattern.

Build pairwise comparison files for:

1. DPO versus SFT.
2. DPO versus base.
3. SFT versus base.
4. Optional: DPO versus GPT-style reference output, if available.

Randomize response order in each pairwise file and preserve the mapping from response position to model identity.

Generation Deliverables

Submit the sanity-check outputs, full generation JSON files for base/SFT/DPO, pairwise JSONL files, generation throughput, and a short note explaining whether you used one or two GPUs.

12 Task 6: Evaluation With Kaggle Models

Use Kaggle model access through `kaggle_benchmarks` or an equivalent Kaggle-supported model interface. A suitable judge model is:

`google/gemini-3.6-flash`

Your evaluator should:

1. Read saved pairwise JSONL files.
2. Judge in batches.
3. Checkpoint judgments after each batch.
4. Sleep briefly between judge requests to avoid hammering the model proxy.
5. Retry failed batches and log unresolved failures.

6. Resume from existing judgments when possible.
7. Summarize win, loss, and tie rates.

The judge should be instructed to compare responses based on helpfulness, correctness, instruction-following, concision when appropriate, formatting, and absence of hallucinations or unsafe content.

Deliverables. Submit judgment JSONL files, summary tables, and a brief discussion of judge reliability.

13 Task 7: Final Report

Your final report should be a complete technical document. It should include:

1. Introduction and motivation.
2. Compute environment and reproducibility details.
3. Dataset preprocessing details.
4. SFT method and results.
5. DPO method and results.
6. Evaluation setup.
7. Quantitative results.
8. Qualitative error analysis.
9. Limitations.
10. Lessons learned.

13.1 Required Tables

Include at least the following tables:

1. SFT training configuration and validation loss.
2. DPO training configuration, validation reward accuracy, and reward margin.
3. Generation throughput for base, SFT, and DPO.
4. Pairwise win/tie/loss rates.

13.2 Required Qualitative Analysis

Discuss examples where:

1. SFT improves over the base model.
2. DPO improves over SFT.
3. DPO is worse than SFT.

4. All models fail.
5. The judge’s decision seems questionable.

14 Expected Runtime

For the recommended 30k SFT, 15k DPO, and 300-prompt evaluation setup, a rough estimate on Kaggle T4x2 is:

Stage	Estimated time
SFT training	1.5 hours
DPO training	2.0–2.5 hours
Generation	0.5–1.0 hours
Kaggle model judging	0.3–0.5 hours
Total	4.5–5.5 hours

Runtime Planning

Budget a six-hour Kaggle session to be safe. Save notebook versions and archive important outputs as soon as a stage completes.

15 Submission Checklist

Submit:

1. GitHub repository URL.
2. Training notebooks or scripts.
3. Root-level `submission_adapter.py` for automated checks.
4. SFT checkpoint path or archive.
5. DPO checkpoint path or archive.
6. AlpacaEval subset file.
7. Base, SFT, and DPO generation files.
8. Pairwise comparison files.
9. Judgment files and summary tables.
10. Final report as PDF.

16 Grading Rubric

Category	Criteria	Points
Repository and reproducibility	Clear repo, documented setup, deterministic subsets, runnable Kaggle workflow	10
SFT implementation	Correct response masking, training loop, validation, checkpointing, stable mixed precision	20
DPO implementation	Correct log-probabilities, DPO loss, reference model handling, validation metrics	25
Evaluation pipeline	Saved generations, pairwise files, Kaggle model judging, summaries	20
Report quality	Clear explanation, quantitative results, qualitative analysis, limitations	20
Engineering hygiene	Disk-aware checkpointing, readable code, useful logs	5
Total		100

17 Academic Integrity and Collaboration

You may discuss concepts, debugging strategies, and experimental observations with classmates. Your implementation and report must be your own. If you use external references, cite them. Do not submit code copied from public DPO implementations as your own. You may consult documentation for PyTorch, Hugging Face Transformers, Kaggle, and the DPO paper.

18 Extension Task: The Idiot Sandwich — TRL, PEFT, and Style Preference Tuning

This extension gives you a second route through preference optimization using higher-level alignment libraries. In the main assignment, you implement DPO yourself. In this extension, you will use TRL’s `DPOTrainer` together with parameter-efficient fine-tuning (PEFT) to train a small instruction model on a curated stylistic preference dataset.

The working title for this task is *The Idiot Sandwich*. The dataset will contain preference pairs that nudge the model toward a Gordon Ramsay-inspired response style: direct, vivid, culinary, impatient, funny, and sharply critical when appropriate. The goal is not to create a model that is abusive or unsafe. The goal is to study how preference optimization can steer *style* as well as helpfulness.

18.1 Model

Use an instruction-tuned SmolLM 350M model. The exact checkpoint will be specified with the dataset release. A typical choice will be a SmolLM 350M Instruct checkpoint from Hugging Face.

This model is intentionally smaller than Qwen/Qwen2.5-0.5B so that students can run multiple hyperparameter settings on Kaggle. Because the model is small, you should expect the style transfer

to be visible but imperfect. Your report should discuss both successful style adoption and failure cases.

18.2 Dataset

You will be given a curated preference dataset with examples of the form:

```
{
  "prompt": "...",
  "chosen": "...",
  "rejected": "..."
}
```

The preferred responses will generally be more aligned with the target persona. The rejected responses may be bland, generic, evasive, poorly formatted, or less stylistically faithful.

The dataset is stylistic, but the model should still remain useful. A good response should:

1. answer the user's actual question;
2. preserve factual correctness;
3. use a Ramsay-inspired tone when appropriate;
4. avoid protected-class insults, threats, harassment, or unsafe content;
5. avoid becoming so performative that it stops being helpful.

18.3 Training Method

Use `TRL's DPOTrainer` rather than your own DPO loop. Use `PEFT/LoRA` so that you train a small set of adapter weights rather than all model parameters.

Your implementation should include:

1. loading the SmolLM 350M Instruct model and tokenizer;
2. formatting the preference dataset for TRL;
3. configuring LoRA with `PEFT`;
4. training with `DPOTrainer`;
5. saving the LoRA adapter and tokenizer;
6. generating samples before and after DPO;
7. evaluating whether the style changed without destroying usefulness.

Unlike the main DPO task, this extension permits TRL and PEFT. You should still write clean, reproducible code and explain each configuration choice.

18.4 Suggested LoRA Configuration

A reasonable starting point is:

Parameter	Suggested value
LoRA rank r	8 or 16
LoRA alpha	16 or 32
LoRA dropout	0.05
Target modules	attention projection modules, or the model-specific common linear modules
Precision	fp16 on T4
Max length	512

You must verify the correct target module names for the SmolLM architecture you use. Do not assume that target module names from Llama, Mistral, or Qwen automatically apply.

18.5 Hyperparameter Sweep

Run a small sweep. The goal is not to exhaustively tune the model, but to observe how DPO and LoRA hyperparameters affect style, helpfulness, and stability.

Hyperparameter	Values
Learning rate	5×10^{-7} , 1×10^{-6} , 2×10^{-6}
β	0.05, 0.1, 0.2
LoRA rank	8, 16, 32
LoRA alpha	16, 32

You do not need to run every possible Cartesian product if compute is limited. Choose a principled subset, such as:

1. one baseline configuration;
2. one lower- β configuration;
3. one higher- β configuration;
4. one larger-rank LoRA configuration.

18.6 Evaluation

Evaluate the base instruct model and your LoRA-DPO variants on a fixed prompt set. The prompt set should include:

1. cooking advice prompts;

2. general instruction-following prompts;
3. prompts where the Ramsay style is appropriate;
4. prompts where excessive harshness would be inappropriate;
5. safety-sensitive prompts where the model should remain responsible.

For each model variant, generate responses and evaluate them along at least three axes:

1. **Helpfulness**: does the response answer the question?
2. **Style adherence**: does it sound like the intended persona?
3. **Safety and restraint**: does it avoid unacceptable insults, harassment, or unsafe guidance?

You may use Kaggle model-based judging for pairwise comparisons. For example, compare:

1. base instruct model versus LoRA-DPO model;
2. low- β versus high- β variants;
3. low-rank versus high-rank LoRA variants.

18.7 Report Requirements for the Extension

Your extension report section should include:

1. model checkpoint used;
2. dataset size and preprocessing details;
3. LoRA configuration;
4. DPO hyperparameters;
5. sweep table;
6. training runtime and GPU memory observations;
7. qualitative samples before and after training;
8. analysis of whether the model became more stylistically faithful;
9. analysis of whether the model lost helpfulness or became too harsh;
10. your recommended configuration and why.

18.8 What to Compare Against the Main Assignment

In your final discussion, compare this extension to your from-scratch DPO implementation:

1. Which was easier to implement and debug?
2. Which was more memory efficient?
3. Which gave clearer metrics?
4. Did LoRA make experimentation easier?

5. Did TRL hide any important details that you now understand because of the from-scratch implementation?

19 Resources

Useful Links

Hugging Face

[Transformers docs](#)

[Qwen2.5-0.5B model card](#)

PEFT / LoRA

[PEFT docs](#)

Preference Optimization

[DPO paper](#)

[TRL docs](#)

[DPOTrainer guide](#)

Kaggle

[Notebooks docs](#)

[Models docs](#)

[GPU usage](#)

References

1. Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. [Direct Preference Optimization: Your Language Model is Secretly a Reward Model](#). NeurIPS, 2023.
2. Ouyang, L. et al. [Training language models to follow instructions with human feedback](#). NeurIPS, 2022.
3. Li, X. et al. [AlpacaEval: An Automatic Evaluator of Instruction-following Models](#). 2023.
4. Bai, Y. et al. [Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback](#). 2022.
5. Hugging Face. [Transformers documentation](#), [TRL documentation](#), and [PEFT documentation](#).
6. Kaggle. [Notebooks documentation](#), [Models documentation](#), and [efficient GPU usage guide](#).